Spring

Prof. Fellipe Guilherme Rey de Souza

DCE701 - Programação Web

Agenda

- O que é?
- Spring vs Jakarta EE
- Spring Boot

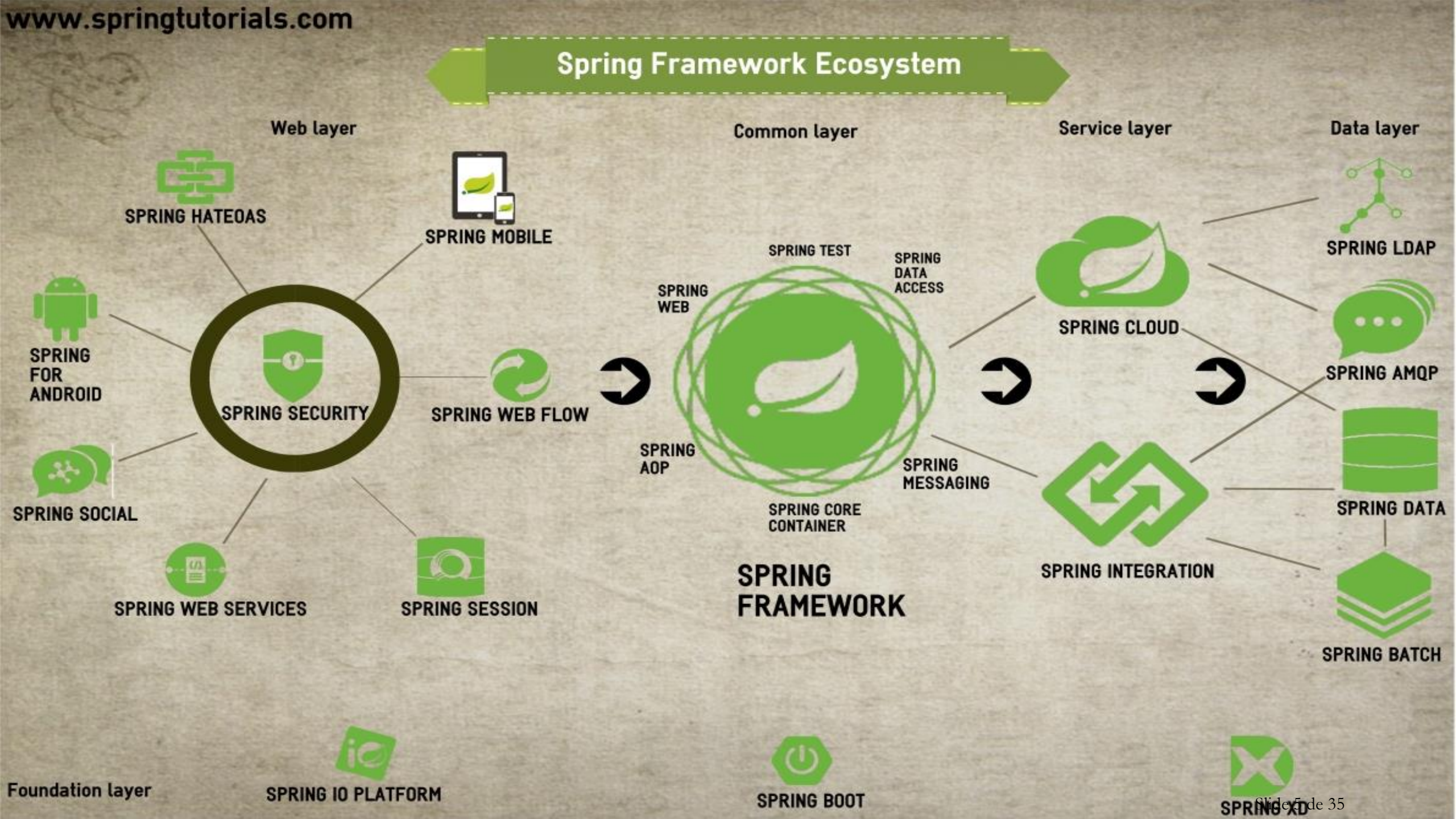


O que é?

Spring – O que é?

- Spring não é apenas um framework backend, mas sim um conjunto de projetos que resolvem vários problemas de um programador backend.

- Ele ajuda a criar aplicações Java com simplicidade e com flexibilidade.
  - Como o Spring tem vários projetos, costumamos nos referir à esses projetos como **Ecossistema Spring**.


www.springtutorials.com
Spring Framework Ecosystem
Web layer
Common layer
Service layer
Data layer
SPRING HATEOAS
SPRING MOBILE
SPRING FOR ANDROID
SPRING SECURITY
SPRING WEB FLOW
SPRING SOCIAL
SPRING WEB SERVICES
SPRING SESSION
SPRING TEST
SPRING DATA ACCESS
SPRING WEB
SPRING AOP
SPRING CORE CONTAINER
SPRING MESSAGING
SPRING FRAMEWORK
SPRING CLOUD
SPRING INTEGRATION
SPRING LDAP
SPRING AMQP
SPRING DATA
SPRING BATCH
Foundation layer
SPRING IO PLATFORM
SPRING BOOT
SPRING XD

Spring – O que é?

- A ideia do Spring é fazer com que o desenvolvedor fique focado nas regras de negócio, de forma que ele não perca tempo desenvolvendo código de infraestrutura da aplicação.
  - Exemplo: Configuração rápida e fácil para conectar com banco de dados relacionais (MySQL, Postgres, etc), banco de dados não relacionais (MongoDB, Redis, etc), messageria (Kafka, RabbitMQ, etc), etc.

Spring – Por que usar o Spring?

- O Spring é um **canivete suíço** para desenvolvedores Java. Ele resolve problemas, como:
  - Gerenciamento de Objetos;
  - Injeção de dependência;
  - Acesso à diferentes banco de dados;
  - REST API;
  - Entre outros!



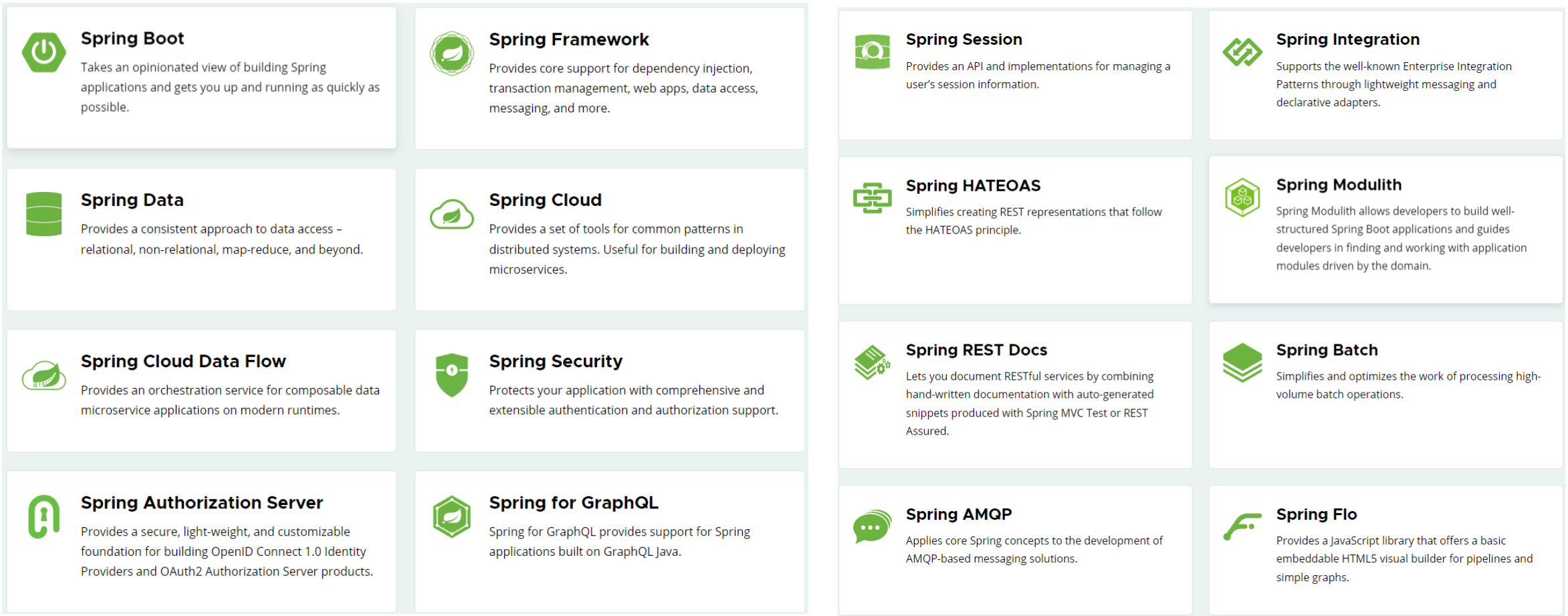
Spring – Por que usar o Spring?

1. Simplifica o desenvolvimento de software
2. Maturidade
3. Projeto modular
4. Em constante evolução
5. Open source
6. Popularidade e empregabilidade

Spring – Quem usa o Spring?

Netflix	TransferWise	Amazon	Cielo
iFood	Santander	Renner	Bank of America
99 Taxi	Itaú	Riachuelo	Walmart
TripAdvisor	JP Morgan	Magazine Luiza	Visa
HotMart	American Express	Lojas Americanas	PayPal
PagSeguro	Mastercard	Netshoes	

Spring – Projetos



Entre outros!

Spring – Primórdios

```xml
<bean id="aroundCall" class = "aop.advice.interceptor.PerformanceInterceptor" />
<bean id="adderImpl" class="aop.advice.target.impl.AdderImpl" />

<bean id = "adder" class = "org.springframework.aop.framework.ProxyFactoryBean">
    <property name = "proxyInterfaces" value="aop.advice.target.Adder" />
    <property name = "interceptorNames">
        <list>
            <value>aroundCall</value>
        </list>
    </property>
    <property name = "target" ref="adderIm
</bean>
```

```xml
<beans:beans xmlns="http://www.springframework.org/schema/security"
  xmlns:beans="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
          http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
          http://www.springframework.org/schema/security
          http://www.springframework.org/schema/security/spring-security-3.1.xsd">

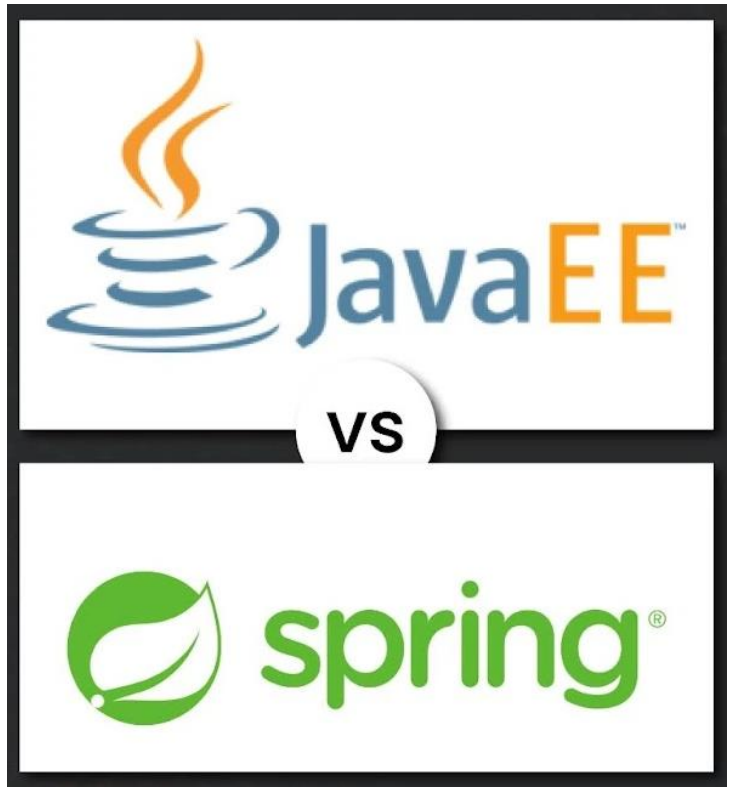
  <beans:bean id="appUserDetailsService" class="com.algaworks.pedidovenda.security.AppUserDetailsService" />

  <http pattern="/javax.faces.resource/**" security="none" />

ssions="true">
ome.xhtml" access="isAuthenticated()" />
edidos/**" access="hasAnyRole('VENDEDORES', 'AUXILIARES', 'ADMINISTRADORES')" />
rodutos/**" access="hasAnyRole('ADMINISTRADORES')" />
*" access="denyAll" />


ser-service-ref="appUserDetailsService">
oder hash="md5" /> -->
```

```xml
<bean id="sessionFactory"
    class="org.springframework.orm.hibernate3.LocalSessionFactoryBean">
    <property name="configLocation"
        value="classpath:hibernate.cfg.xml">
    </property>
</bean>
<bean id="PersistanceLayer" class="com.hibernate.PersistanceLayer">
    <property ref="SubjectDAO" name="subjectDAO"/>
    <property ref="CategoryDAO" name="categoryDAO"/>
    <property ref="UserDAO" name="userDAO"/>
    <property ref="ModeratorDAO" name="moderatorDAO"/>
    <property ref="PostDAO" name="postDAO"/>
</bean>
<bean id="SubjectDAO" class="com.hibernate.SubjectDAO">
    <property name="sessionFactory">
        <ref bean="sessionFactory" />
    </property>
</bean>
```



Qual a diferença? Qual é melhor?

Spring *vs* Jakarta EE

- O Spring Framework e o Jakarta EE (anteriormente conhecido como Java EE) são dois frameworks Java amplamente utilizados para o desenvolvimento de aplicativos empresariais.

- Embora ambos tenham objetivos similares, existem algumas diferenças significativas entre eles.

Spring *vs* Jakarta EE – Origem

- **Spring**: Desenvolvido pela SpringSource (agora parte da VMware), o Spring surgiu como uma alternativa ao modelo de programação Java EE, inicialmente focando em resolver problemas de complexidade e configuração do Java EE.

- **Jakarta EE**: É um padrão da Oracle para o desenvolvimento de aplicativos empresariais Java. A comunidade Jakarta EE agora é gerida pela Eclipse Foundation, após a mudança da Oracle para a Eclipse Foundation.

Spring *vs* Jakarta EE – Arquitetura

- **Spring**: Adota uma abordagem modular e leve, permitindo que os desenvolvedores escolham os módulos específicos do Spring que desejam usar. É conhecido por sua flexibilidade e facilidade de integração com outras tecnologias.

- **Jakarta EE**: Baseia-se em especificações mais rígidas e padronizadas, proporcionando uma plataforma mais monolítica. Jakarta EE inclui um conjunto de APIs e especificações para serviços empresariais, como servlets, EJB (Enterprise JavaBeans), JPA (Java Persistence API), entre outros.

Spring *vs* Jakarta EE – DI

- **Spring**: Conhecido por seu contêiner de inversão de controle (IoC) e injeção de dependência robustos, o Spring oferece uma ampla flexibilidade na configuração e gerenciamento de componentes.

- **Jakarta EE**: Tradicionalmente, dependia muito do EJB para injeção de dependência, mas com a evolução das versões, o CDI (Contexts and Dependency Injection) tornou-se a principal tecnologia para injeção de dependência em Jakarta EE.

Spring *vs* Jakarta EE – Configuração

- **Spring**: Utiliza uma abordagem baseada em código ou configuração XML, mas recentemente favorece configuração baseada em anotações (como @Autowired) e configuração por meio do Java.

- **Jakarta EE**: Historicamente dependia fortemente de configuração XML, mas com o avanço das versões, tem adotado mais configuração baseada em anotações, especialmente com a adoção do CDI.

Spring *vs* Jakarta EE – Configuração

- **Spring**: Adota uma licença Apache, o que significa uma licença de código aberto mais permissiva.

- **Jakarta EE**: Também utiliza licenças de código aberto, mas a escolha exata pode variar entre as implementações específicas.

Spring *vs* Jakarta EE

- A escolha entre Spring e Jakarta EE muitas vezes depende das preferências da equipe de desenvolvimento, dos requisitos específicos do projeto e do histórico de tecnologias já adotadas pela organização.

- Algumas empresas podem até optar por uma abordagem híbrida, usando partes do Spring e do Jakarta EE conforme necessário.



O que é o projeto Spring Boot?

Spring Boot – Visão Geral

- O Spring Boot é uma extensão do Spring Framework que simplifica significativamente o processo de desenvolvimento de aplicativos Java.

- Ele fornece uma maneira rápida e fácil de criar aplicativos autônomos, prontos para produção, com a mínima configuração possível.

Spring Boot – Visão Geral

- O principal objetivo do Spring Boot é facilitar o desenvolvimento de aplicativos Spring, eliminando grande parte da complexidade associada à configuração e à inicialização de um projeto Spring.
  - Para isso, ele usa um conceito chamado de **Convention Over Configuration** – ou, em português, **Convenção sobre Configuração**.

Spring Boot – Características

- **Convenção sobre Configuração**:
  - Spring Boot adota a filosofia de "convenção sobre configuração", o que significa que ele fornece configurações padrão sensatas para a maioria dos casos.
  - Isso reduz a necessidade de configuração manual e permite que os desenvolvedores se concentrem mais na lógica do aplicativo do que na configuração.

Spring Boot – Características

- **Starter POMs**:
  - Os Starter POMs (Project Object Models) são uma característica distintiva do Spring Boot.
  - Eles são artefatos Maven preconfigurados que simplificam a inclusão de dependências comuns em seu projeto, proporcionando uma inicialização rápida.

Spring Boot – Características

- **Autoconfiguração:**
  - O Spring Boot inclui uma funcionalidade de autoconfiguração que analisa o ambiente em que o aplicativo está sendo executado e automaticamente configura componentes do Spring com base nesse ambiente.
  - Isso significa menos configuração manual e menos código boilerplate.

Spring Boot – Características

- **Desenvolvimento de Aplicativos Autônomos:**
  - Spring Boot permite a criação de aplicativos autônomos (standalone) que podem ser executados sem a necessidade de um servidor de aplicativos externo.
  - Isso é possível graças à inclusão de servidores embutidos, como Tomcat, Jetty ou Undertow.

Spring Boot – Características

- **Monitoramento e Gerenciamento:**
  - O Spring Boot Actuator oferece recursos de monitoramento e gerenciamento prontos para uso, como endpoints HTTP para informações de saúde, métricas, informações do ambiente e muito mais.

Spring Boot – Características

- **Integração com o ecossistema Spring:**
  - Spring Boot é totalmente compatível com o ecossistema Spring existente, incluindo Spring Data, Spring Security, Spring Cloud, entre outros.
  - Portanto, os desenvolvedores podem aproveitar as vantagens desses projetos ao criar aplicativos Spring Boot.

Spring Boot – Características

- **Fácil Integração com Frameworks Front-End:**
  - Facilita a integração com frameworks front-end, como Angular, React ou Vue.js, permitindo o desenvolvimento de aplicativos web modernos.

spring-io/**initializr**

A quickstart generator for Spring projects



96 Contributors | 48 Issues | 3k Stars | 2k Forks





Como iniciar um projeto?

Spring Boot – Iniciando um projeto

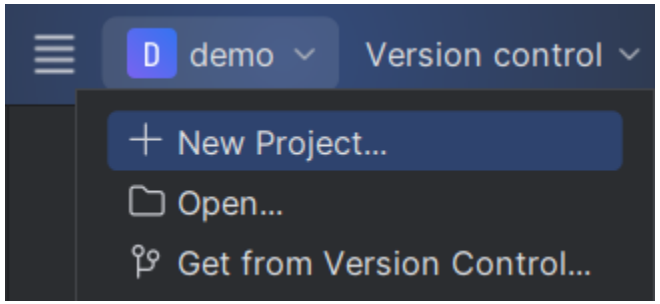
- O **Spring Initializr** é uma ferramenta web que simplifica o processo de inicialização e configuração de projetos Spring.

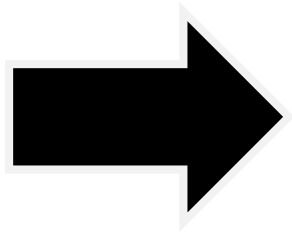
- Ele fornece uma interface gráfica fácil de usar para gerar um esqueleto de projeto com as dependências e configurações iniciais necessárias para começar a desenvolver com o Spring Framework ou o Spring Boot.

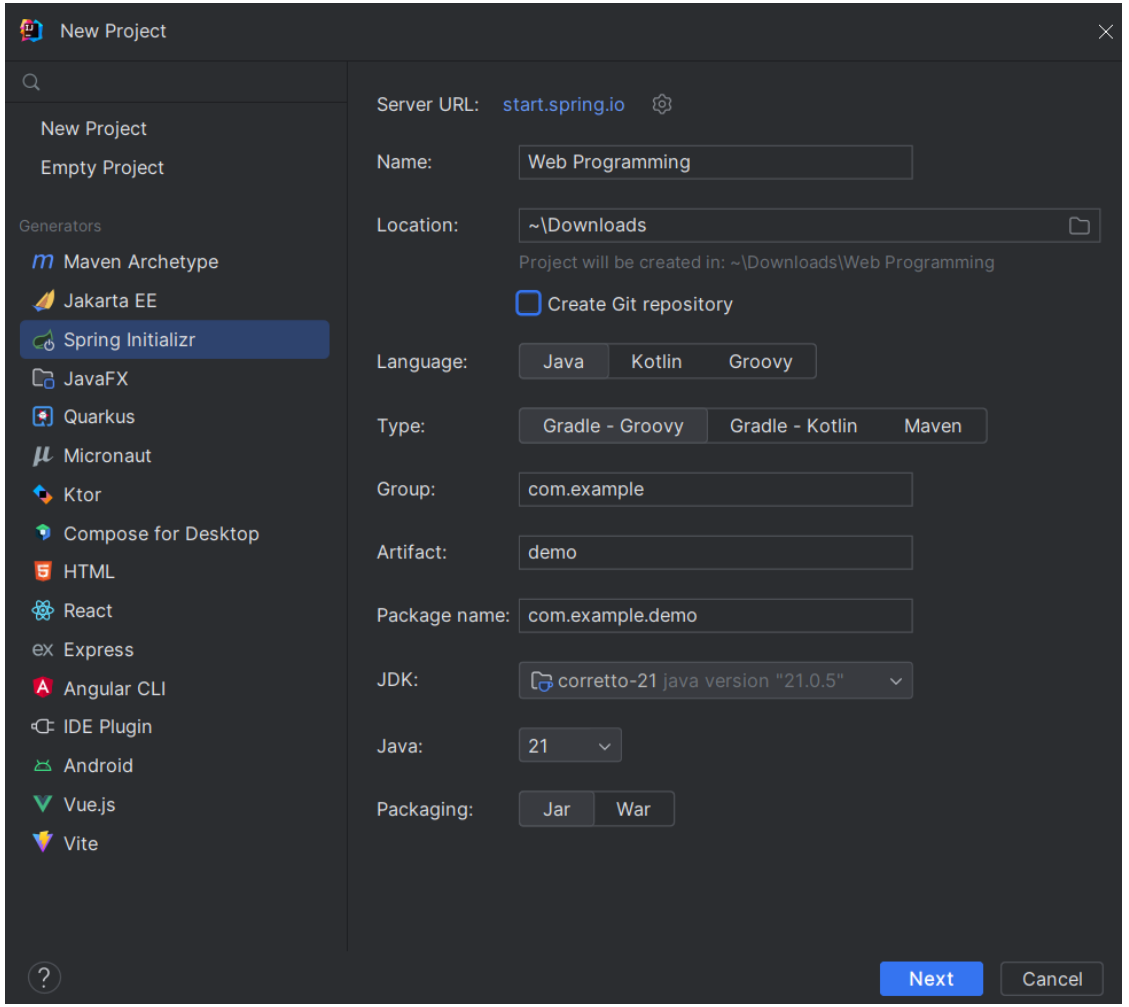
Spring Boot – Iniciando um projeto

- Existem duas principais formas de se iniciar um projeto Spring Boot usando o **Spring Initializr**:
  1. Através da IDE **IntelliJ**
     - Supondo que esta seja a IDE a ser utilizada.
  2. Através do **site** do Spring Initializr
     - Disponível em: https://start.spring.io/

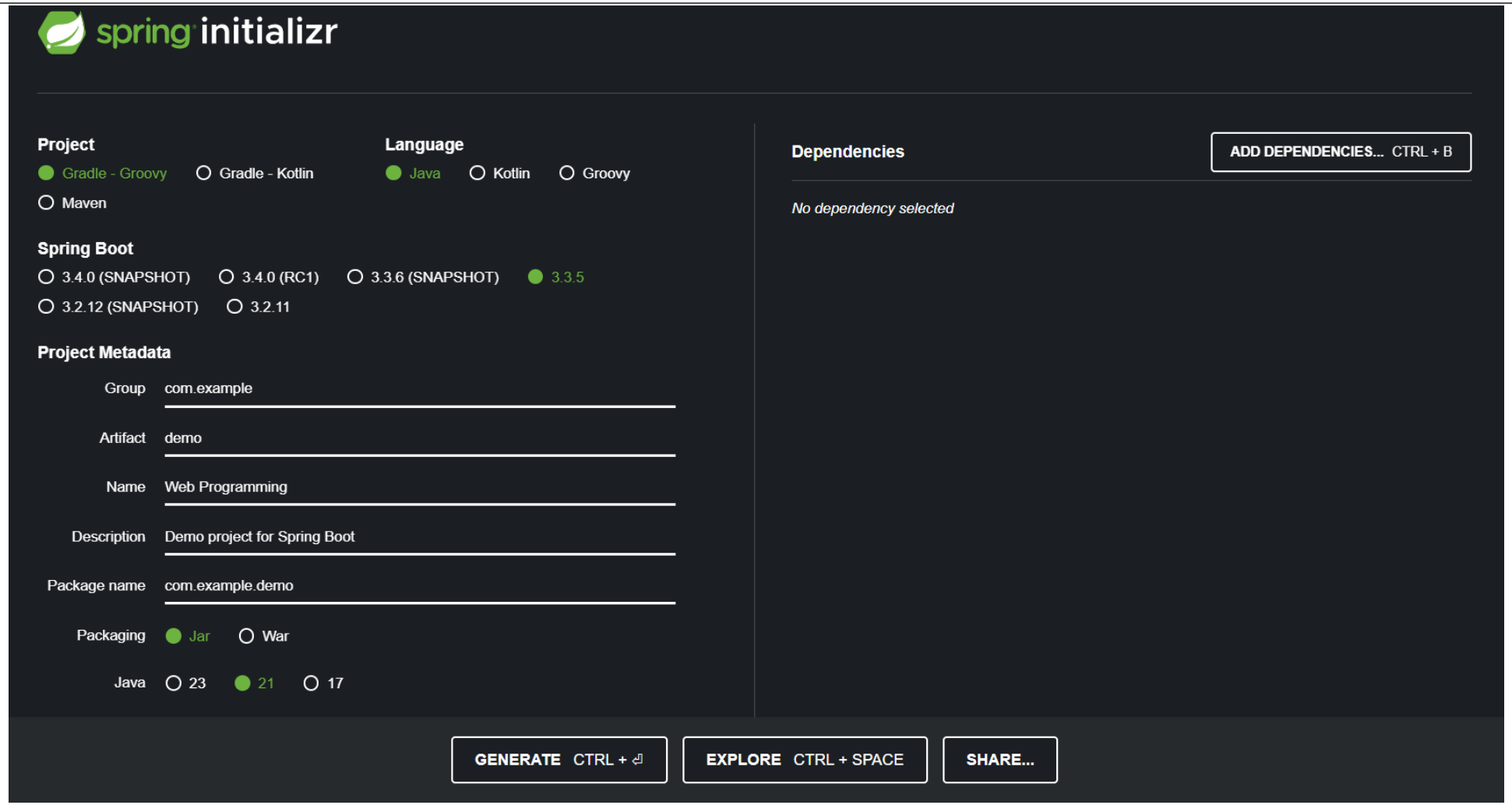
Spring Boot – Iniciando um projeto







Spring Boot – Iniciando um projeto



Spring

Prof. Fellipe Guilherme Rey de Souza

DCE701 - Programação Web